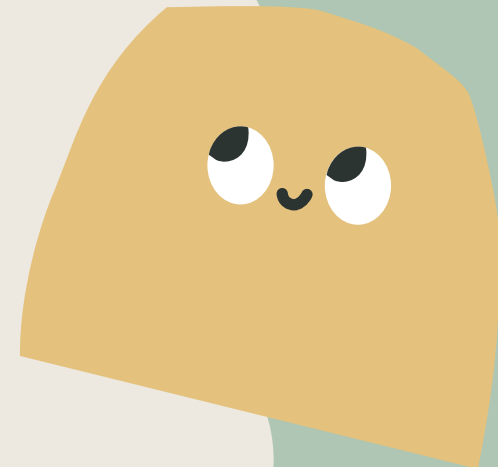
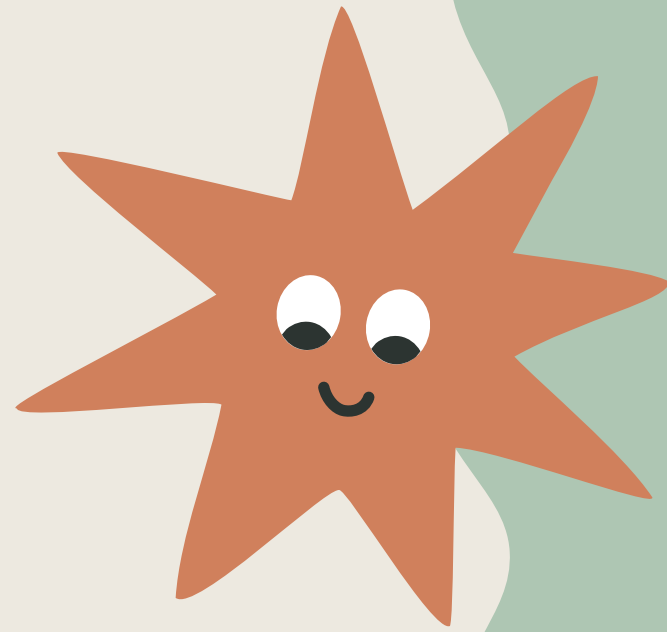


Big O Examples & Complexity Analysis in C++

Lecture 5



Learning Outcomes (Lecture 5)

- At the end of this lecture, students will be able to:
 - Analyze the time complexity of C++ programs.
 - Apply operation counting to derive **$T(n)$** .
 - Use summation formulas in complexity analysis.
 - Analyze logarithmic loops.
 - Analyze mixed loops.
 - Solve university exam-style complexity questions

Example 1 – Triangular Loop ($\sum i$)

➤ Step 1:

- Outer loop = n
- Inner Loop = $1 + 2 + 3 + 4 + \dots + n$

➤ Step 2:

- $T(n) = n + (1 + 2 + 3 + \dots + n)$
- $T(n) = n + \sum i$
- $T(n) = n + n(n + 1) / 2$
- $T(n) = n + (n^2 + n) / 2$
- $T(n) = n + n^2/2 + n/2$

➤ Step 3:

- $O(n^2)$

```
int sum = 0;

for(int i = 1; i <= n; i++)
{
    for(int j = 1; j <= i; j++)
    {
        sum += j;
    }
}
```

Example 2 – $\sum i^2$

➤ Step1:

- Outer loop = n
- Inner Loop = $\sum i^2$

➤ Step 2:

- $T(n) = \sum i^2$
- $T(n) = n(n+1)(2n+1)/6$
- $T(n) = n(2n^2 + 3n + 1) / 6$
- $T(n) = (2n^3 + 3n^2 + n) / 6$

➤ Step 3:

- $O(n^3)$

```
long long total = 0;

for(int i = 1; i <= n; i++)
{
    for(int j = 1; j <= i * i; j++)
    {
        total += j;
    }
}
```

Example 3 – Logarithmic Loop

➤ Step1: Analyze the Loop

- Previous loops $i++$
- This loop $i *= 2$

➤ Step 2: Dry Run

- Suppose $n = 20$

Iteration	i
1	1
2	2
3	4
4	8
5	16
6	32 (Stop)

```
int sum = 0;

for(int i = 1; i < n; i *= 2)
{
    sum += i;
}
```

Example 3 – Logarithmic Loop

➤ Step 2: Dry Run

- Suppose $n = 100$

Iteration	i
1	1
2	2
3	4
4	8
5	16
6	32
7	64
8	128 (Stop)

```
int sum = 0;

for(int i = 1; i < n; i *= 2)
{
    sum += i;
}
```

Example 3 – Logarithmic Loop

➤ Step 3: Identify Pattern

Iteration	i	Pattern
1	1	2^0
2	2	2^1
3	4	2^2
4	8	2^3
5	16	2^4
6	32	2^5
7	64	2^6
8	128 (Stop)	2^7

```
int sum = 0;

for(int i = 1; i < n; i *= 2)
{
    sum += i;
}
```

Example 3 – Logarithmic Loop

➤ Step 4: General Formula

- Suppose loop runs k times, then
- $i = 2^k$

➤ Step 5: when does the loop stop

- When $i \geq n$
- Since $i = 2^k$
- We get $2^k = n$
- How do we find k from $2^k = n$

```
int sum = 0;

for(int i = 1; i < n; i *= 2)
{
    sum += i;
}
```

Example 3 – Logarithmic Loop

➤ Step 6: Logarithm

- Taking log of both sides
- $\log_2(2^k) = \log_2(n)$
- Using the logarithm rule
- $k = \log_2(n)$

➤ Step 7: Final Answer

- Since loop executes $\log_2(n)$ times
- $T(n) = \log_2(n)$
- Ignoring the base
- $O(n) = \log n$

```
int sum = 0;

for(int i = 1; i < n; i *= 2)
{
    sum += i;
}
```

Example 4 – Halving Loop

➤ Step1:

- Identify increment/decrement statement
- $i /= 2$
- The loop variable is **halved** every iteration

➤ Step 2:

- We know that
- If a loop variable is **multiplied or divided by a constant (>1)** in each iteration, the loop executes **$\log n$** times

➤ Step 3:

$$T(n) = \log n$$

➤ Step 4:

$$O(\log n)$$

```
int sum = 0;

for(int i = n; i > 1; i /= 2)
{
    sum += i;
}
```

Example 5 – Halving Loop

➤ Step 1:

- Outer loop executes **$\log n$** times

➤ Step 2:

- Inner loop executes **n** times

➤ Step 3:

- Since the loops are nested, multiply the complexities
- $T(n) = \log n \times n$

➤ Step 4:

- $O(n \log n)$

```
int sum = 0;

for(int i = 1; i < n; i *= 2)
{
    for(int j = 0; j < n; j++)
    {
        sum += j;
    }
}
```

Example 6

➤ Step1:

- Outer loop executes **n - 4** times = $O(n)$

➤ Step 2:

- Inner loop executes **4** times = $O(1)$

➤ Step 3:

- Since the loops are nested, multiply the complexities
- $O(n) \times O(1)$
- $O(n)$

```
for (i = 4; i < n; i++)  
{  
    for (j = i - 3; j <= i; j++)  
        sum += a[j];  
  
    cout << "sum = " << sum << endl;  
}
```

Example 6

```
for (i = 4; i < n; i++)  
{  
    for (j = i - 3; j <= i; j++)  
        sum += a[j];  
  
    cout << "sum = " << sum << endl;  
}
```

$$1 + 4(n - 4)$$

$$4 \times 3 = 12$$

$$T(n) = 13 + 4(n - 4)$$

$$O(n)$$